Project - High Level Design for CI/CD for Go (Golang) Logistics App

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	PRAKRATI RASANIYA	EN22CS301722
2	PARTH AKHAND	EN22CS301687
3	REET SOOD	EN22CS301796
4	ROSHNI RAWAT	EN22CS301826
5	RIYANSHI JAIN	EN22CS301816
6	RISHI YADAV	EN22CS301806

Group Name: GROUP 02D7

Project Number: DO-25

Industry Mentor Name: Aashruti Shah Ma'am

University Mentor Name: Prof Shyam Patel

Academic Year: 2022-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The High Level Design (HLD) outlines a CI/CD pipeline for a Go-based Logistics App, automating build, test, and deployment processes to ensure reliable releases. It assumes a containerized microservices architecture typical for logistics systems handling orders, tracking, and inventory

1.1 Scope of the document

- Covers overall architecture and CI/CD pipeline design.
- Includes tools like Docker, GitHub Actions, and Kubernetes.
- Describes deployment and integration workflow.
- Defines system boundaries and major components.
- Excludes low-level implementation details.

1.2 Intended Audience

- Developers working on backend and APIs.
- DevOps engineers managing deployment pipeline.
- System architects designing infrastructure.
- Academic evaluators reviewing the project.
- Anyone needing high-level system understanding.

1.3 System Overview

- Golang-based logistics application for shipment management.
- Provides REST APIs for system interaction.
- Uses Docker for containerization.
- GitHub Actions automates CI/CD pipeline.
- Designed for scalable and reliable deployments.

2. System Design

- Defines architecture and workflow of the system.
- Includes application and pipeline structure.
- Explains interaction between components.
- Focuses on automation and deployment.
- Ensures system scalability and efficiency.

2.1 Application Design

- Built using Golang with modular architecture.
- Exposes REST APIs for logistics operations.
- Ensures high performance and scalability.
- Easy to maintain and extend.
- Follows clean coding practices.

2.2 Process Flow

- Developer writes and pushes code to GitHub.
- CI/CD pipeline is triggered automatically.
- Code is built, tested, and validated.
- Docker image is created.
- Application is deployed to server/Kubernetes.

2.3 Information Flow

- User sends request to API.
- Request processed by Golang service.
- Data is fetched/stored in database.
- Response is generated.
- Output returned to user.

2.4 Components Design

- Golang application for backend logic.
- Docker for containerization.
- GitHub Actions for CI/CD automation.
- Kubernetes for orchestration.
- Database for storing logistics data.

2.5 Key Design Considerations

- Scalability to handle increasing users.
- Reliability for consistent performance.
- Security of data and APIs.
- Automation to reduce manual work.
- Maintainability for future upgrades.

2.6 API Catalogue

- /health endpoint for system status.
- /shipments for creating shipment records.
- APIs follow REST principles.
- JSON used for data exchange.
- Easy integration with frontend systems.

3. Data Design

- Defines how logistics data is stored.
- Includes shipment and system logs.
- Ensures data consistency.
- Supports future scalability.
- Maintains structured format.

4. Interfaces

- REST APIs for client interaction.
- CI/CD tools integration.
- Communication between services.
- Supports external systems.
- Standard protocols used.

5. State and Session Management

- Application is stateless.
- No session stored on server.
- Improves scalability.
- Simplifies deployment.
- Easy load balancing.

6. Caching

- Redis used for caching.
- Reduces database load.
- Improves response time.
- Stores frequently used data.
- Enhances performance.

7. Non-Functional Requirements

- Includes performance and security needs.
- Ensures system availability.
- Handles scalability requirements.
- Provides reliability.
- Supports efficient operation.

8. References

• OpenAI. (2023). GPT-4 Technical Report. Explains the architecture and capabilities of large language models used for reasoning and generation.

• Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models

• Wu, B., et al. (2023). AutoGPT: Autonomous GPT-4 Experiment. A reference for autonomous agent systems that perform multi-step tasks.

• LangChain Documentation. (2024). Official documentation explaining tool calling, memory, chains, and agent workflows.

• LangGraph Documentation. (2024). Used for orchestration and graph-based agent execution.